Standard Operating Procedure (SOP)

REAL-TIME HAND GESTURE ENGINE & ARDUINO LED INTERFACE

1. Purpose & Project Scope

This document outlines the step-by-step technical procedures required to install, configure, and deploy a real-time computer vision finger-counting pipeline from an absolute clean slate. The completed environment tracks hand orientation dynamics and transmits active spatial configurations over serial connection vectors to command physical breadboard LEDs.

2. Phase 1: Environment Setup & Core Dependencies

Step 2.1: Python Runtime Installation

1. Download the system package executable installer directly from the official web repository: `python.org/downloads`.
2. Run the downloaded installer initialization wizard.
3. **CRITICAL WARNING:** You must click the checkbox marked **"Add python.exe to PATH"** at the bottom of the opening panel window. If this configuration is omitted, your computer's terminal shells will fail to compile or locate runtime paths.
4. Select **Install Now**, then close the dashboard interface when finished.

Step 2.2: Verify Environmental Variables

Launch a new terminal instance (press Windows Key, type `cmd`, and hit Enter) and input the following query commands to verify system status:

```
python --version
pip --version
```

The command output string must report active installation index records (e.g., Python 3.10.x). Do not progress to library configuration if these commands report path location exceptions.

Step 2.3: Retrieve Package Dependencies

Run the command listed below within your active terminal frame window interface to safely pull and unpack the necessary functional image-processing models over the network:

```
pip install opencv-python cvzone pyserial
```

3. Phase 2: Hardware Assembly Configuration

Set up your workspace prototype breadboard layout using an Arduino development board wired matching this explicit functional architecture:

- **LED Pin 1 (Thumb tracking state):** Microcontroller Digital **Pin 2**
- **LED Pin 2 (Index tracking state):** Microcontroller Digital **Pin 3**
- **LED Pin 3 (Middle tracking state):** Microcontroller Digital **Pin 4**
- **LED Pin 4 (Ring tracking state):** Microcontroller Digital **Pin 5**
- **LED Pin 5 (Pinky tracking state):** Microcontroller Digital **Pin 6**
- **In-Line Resistive Elements:** Bridge a 220 Ohm or 330 Ohm current-limiting resistor inline between the target digital pin terminal and the anode (longer positive pin) of each LED to prevent overcurrent.
- **Common Ground Reference:** Connect the short negative cathode legs of all 5 LEDs together to a shared vertical terminal track on the breadboard, running a single return wire line directly into an Arduino **GND** socket.

4. Phase 3: Software Source Registries

Initialize your local project storage registry directory folder (e.g., `C:\Users\Ravi-pc\FingureCount`).

A. Microprocessor Firmware Code (led_control.ino)

Open your desktop Arduino IDE application instance dashboard, copy this program logic, and flash it to your connected chip. **Close out the active Arduino IDE Serial Monitor window directly following a successful upload.**

```
// Map physical LEDs to consecutive digital pins
const int ledPins[5] = {2, 3, 4, 5, 6};

void setup() {
  // Configure pins as low-impedance output lines
  for (int i = 0; i < 5; i++) {
    pinMode(ledPins[i], OUTPUT);
    digitalWrite(ledPins[i], LOW); // Initialize with low states (LEDs Off)
  }

  // Initialize Serial UART connection pipe running at 9600 Baud rate
  Serial.begin(9600);
}

void loop() {
  // Execute logical parsing loops only when receiving a complete 5-character string
  if (Serial.available() >= 5) {

    for (int i = 0; i < 5; i++) {
      char state = Serial.read(); // Read current indexed character byte position

      if (state == '1') {
```

```
        digitalWrite(ledPins[i], HIGH); // Assert high logic level -> ON
      } else if (state == '0') {
        digitalWrite(ledPins[i], LOW);  // Assert low logic level -> OFF
      }
    }

    // Clear out residual streaming buffers to protect real-time pipeline sync
    while(Serial.available() > 0) {
      Serial.read();
    }
  }
}
```

B. Main Vision Automation Pipeline Script (code.py)

Save this runtime script block directly as `code.py` within your designated project directory:

```
import cv2
from cvzone.HandTrackingModule import HandDetector
import serial
import time

# 1. Establish Secure USB Serial Data Pipe Hook
try:
    # IMPORTANT: Modify 'COM3' to match your explicit device port registration
    arduino = serial.Serial(port='COM3', baudrate=9600, timeout=0.1)
    time.sleep(2) # Enforce a 2-second setup lock allowing internal chip resets
    print("Microcontroller communications pipeline verified!")
except Exception as e:
    print(f"Serial Interconnect Notice: {e}\nInitializing in local software emulation
mode...")
    arduino = None

# 2. Configure Local Web Camera Stream Capture
cap = cv2.VideoCapture(0)
detector = HandDetector(detectionCon=0.8, maxHands=1)

while True:
    success, img = cap.read()
    if not success:
        break

    # Extract multidimensional hand joint arrays
    hands, img = detector.findHands(img)

    if hands:
        hand1 = hands[0]
        lmList = hand1["lmList"] # Vector dictionary representing 21 distinct mapping
positions

        fingers = []

        # Track 4 standard fingers [Index, Middle, Ring, Pinky] via Y-axis values
        fingers.append(1 if lmList[8][1] < lmList[6][1] else 0)   # Index Tip vs Knuckle
        fingers.append(1 if lmList[12][1] < lmList[10][1] else 0) # Middle Tip vs Knuckle
        fingers.append(1 if lmList[16][1] < lmList[14][1] else 0) # Ring Tip vs Knuckle
```

```
        fingers.append(1 if lmList[20][1] < lmList[18][1] else 0) # Pinky Tip vs Knuckle

        # Universal Absolute-Distance Thumb Tracking Rule (Bypasses hand mirroring)
        thumb_tip = lmList[4]
        index_base = lmList[5]
        distance = abs(thumb_tip[0] - index_base[0])

        # Track if the horizontal spread is open or closed
        if distance > 35:
            fingers.insert(0, 1)  # Stretched state -> Open
        else:
            fingers.insert(0, 0)  # Tucked state -> Closed

        # Package state array into a transmission string (e.g. "11100")
        finger_string = "".join(map(str, fingers))
        total_active = fingers.count(1)

        # Draw text output strings directly over local pixel arrays
        cv2.putText(img, f'Fingers: {total_active} ({finger_string})', (50, 50),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

        # Transmit the data stream to your Arduino over USB
        if arduino:
            arduino.write(bytes(finger_string, 'utf-8'))

    else:
        # Emergency default state: Clear hardware arrays if hand context drops out of
frame
        if arduino:
            arduino.write(bytes("00000", 'utf-8'))

    cv2.imshow("Universal Finger Counter", img)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

if arduino:
    arduino.close()
cap.release()
cv2.destroyAllWindows()
```

5. Phase 4: Active Runtime Execution Steps

1. Verify all electronics circuit wiring maps match Section 3 exactly, and lock your microcontroller connection via its USB interface cable line.
2. Launch your command terminal application dashboard window and relocate its operating focus to your physical project folder path:
   `cd C:\Users\Ravi-pc\FingureCount`
3. Initialize the core control interpreter script engine loop:
   `python code.py`
4. Place either your left or right hand inside the open webcam frame field. Changing your extended finger joints will immediately drive your hardware LEDs to toggle high or low matching your gestures.

5. Focus the live tracking video view window frame panel and hit the `q` key line configuration on your keyboard to cleanly disengage all active serial stream hooks.